

Virtual Environments

What a `.venv` directory actually is, why every serious Python project has one, and how it quietly redirects the import search path so each project sees only its own dependencies.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 12 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Dependency isolation · site-packages · `.venv` · python symlink · activation · PATH · pip ·
Dependency hell · Reproducibility

Table of Contents

Chapter 12 — Virtual Environments	3
12.1 The problem: one shared pile of packages	3
12.2 What a .venv actually is	4
12.3 What "activation" really does	5
12.4 Installing into an environment, and reproducibility	6
12.5 Bridge — environments around hathor-core	6
Recap	7

Chapter 12 — Virtual Environments

What you'll learn in this chapter

- The problem virtual environments solve — **dependency hell** — and why installing packages "globally" breaks down the moment you have two projects.
- What a `.venv` actually is: not magic, but a directory holding its own Python, its own `site-packages`, and a few scripts.
- Where third-party packages physically live (**site-packages**) and how that connects to the import search path from Chapter 11.
- What "**activating**" an environment really does (it's mostly one change to `PATH`), and why you can ignore activation entirely if you address the environment's Python directly.
- How **pip** installs into an environment, and how this all sets up **Poetry** (Chapter 13), which manages an environment for you.
- A **bridge** to how `hathor-core` is run inside one.

Chapter 11 ended at the import search path, `sys.path`, and noted that one entry on it — where installed packages live — is what a virtual environment manipulates. This chapter opens that box. A virtual environment¹ is one of those tools that feels mysterious until you look inside, at which point it becomes obvious and even a little mundane — which is exactly the understanding you want. You have seen `.venv` directories and `source .../activate` incantations; by the end of this chapter you'll know precisely what they are and why `hathor-core`, like every serious Python project, lives inside one.

This is a short \$5-style primer: the problem, what the thing is, how it works, how Hathor uses it, and why it beats the alternatives.

12.1 The problem: one shared pile of packages

When you install Python, it comes with one global location where third-party packages go — the system **site-packages**² directory. Run `pip install requests` without any environment, and `requests` lands in that one shared pile, available to every Python program on the machine. For a single project, fine. The trouble starts with the second project.

Picture two projects on one machine:

- **Project A** needs `twisted` version 22 and `pydantic` version 1.

- **Project B** needs `twisted` version 24 and `pydantic` version 2.

With one shared site-packages, there is exactly one installed version of `twisted` and one of `pydantic` for both. Install what B needs and you break A; reinstall for A and you break B. They cannot coexist. This is **dependency hell**³: independent projects fighting over a single global namespace of package versions. It gets worse with transitive dependencies (the packages your packages need), where two libraries demand incompatible versions of a third.

Three further pains compound it:

- **Polluting the system Python.** Your operating system itself often uses Python for its own tools. Installing or upgrading packages globally can break those, sometimes badly.
- **No reproducibility.** "It works on my machine" — because your global pile happens to have the right versions. A teammate's differs, and the code behaves differently or won't run.
- **No clean uninstall.** A project's dependencies are tangled into the global pile with everyone else's; you can't cleanly remove just one project's footprint.

The fix is isolation: give each project its own private pile of packages, walled off from the system and from other projects. That private pile is a virtual environment.

12.2 What a `.venv` actually is

Here is the demystifying truth: **a virtual environment is just a directory.**

Conventionally named `.venv` or `venv`, it contains a self-sufficient, isolated Python setup for one project. Look inside and you find three things that matter:

```
.venv/
├── bin/                (Scripts/ on Windows)
│   ├── python         → a symlink (or tiny copy) pointing at a real interpreter
│   ├── pip
│   └── activate        → a script that tweaks your shell (see §12.3)
├── lib/
│   └── python3.x/
│       └── site-packages/ ← THIS project's private package pile
└── pyvenv.cfg          → a small config naming the base interpreter
```

The two load-bearing parts:

1. **Its own python.** The `bin/python` inside the environment is a **symlink**⁴ (or thin wrapper) back to a real Python interpreter installed on your system — the environment does not contain a whole copy of Python. But when you run that `python`, it is configured to use the environment's own `site-packages`, not the global one.

2. **Its own `site-packages`**. This is the private pile from §12.1. Packages installed into this environment land here and nowhere else. Project A's environment has `twisted` 22 here; Project B's has `twisted` 24 in its separate `.venv/lib/.../site-packages`. They never collide, because they are different directories.

That is the whole trick, and it connects straight back to Chapter 11: **the environment's `python` puts its own `site-packages` on `sys.path` instead of the global one**. Recall that `import` searches `sys.path` and the first match wins (Chapter 11 §11.5). A virtual environment is, at bottom, a mechanism for swapping which `site-packages` directory is on the import search path. Everything else — activation, the scripts — is convenience around that one idea.

```
running the GLOBAL python:    sys.path includes /usr/lib/.../site-packages
running .venv/bin/python:    sys.path includes .venv/lib/.../site-packages
                             (the global one is excluded)
→ same `import twisted`, different twisted, decided by WHICH python you ran
```

12.3 What "activation" really does

You'll see instructions to activate an environment: `source .venv/bin/activate`. This sounds like flipping a switch deep in Python, but it is almost entirely a shell convenience. Activation mainly does one thing: it **prepends the environment's `bin/` directory to your shell's `PATH`**⁵.

`PATH` is the ordered list of directories your shell searches when you type a command name. By putting `.venv/bin` at the front, activation ensures that when you type `python` or `pip`, the shell finds the environment's `python/pip` first — so commands "just use" the environment without your typing its full path. (It also sets a couple of environment variables and usually changes your prompt to show the active environment's name, so you don't lose track of which one you're in.)

The clarifying consequence: **activation is optional**. Because the real isolation lives in which `python` binary runs (§12.2), you can skip activation entirely and address the environment directly:

```
# with activation:
source .venv/bin/activate
python run_node.py           # "python" now means .venv's python

# without activation – identical effect:
.venv/bin/python run_node.py # address the environment's python directly
```

Both run the same interpreter with the same private `site-packages`. Activation is just so you can type `python` instead of `.venv/bin/python` repeatedly. This matters in practice because **tools that manage environments for you** — Poetry (Chapter 13), Docker (Chapter 15) — often run the environment's Python directly and never "activate" anything in the interactive sense. Understanding that activation is cosmetic keeps those tools from seeming mysterious.

12.4 Installing into an environment, and reproducibility

Inside an environment, **pip**⁶ — Python's package installer — fetches packages (from the Python Package Index, **PyPI**⁷) and places them in this environment's `site-packages`. `pip install twisted` while environment A is active installs Twisted into A alone.

This gives back everything dependency hell took away:

- **Isolation:** each project's versions are independent; A's Twisted 22 and B's Twisted 24 coexist on one machine, in separate environments.
- **A clean system Python:** you never touch the global pile, so OS tools stay safe.
- **Reproducibility:** because an environment is built from an explicit list of packages, a teammate (or a server, or a Docker image) can build an identical environment from the same list — "works on my machine" becomes "works in every environment built from this spec."

That last point is where plain pip shows its limit, and where the next chapter enters. A bare `pip install twisted` installs whatever the latest compatible version is today — which may differ next week, breaking reproducibility. Serious projects therefore record the exact versions of every package (and every transitive dependency) in a **lock file**, and use a tool that installs precisely those. That tool, for `hathor-core`, is **Poetry** — Chapter 13. A virtual environment is the where (an isolated pile); Poetry is the what and which version (a managed, locked, reproducible set of packages installed into that pile).

12.5 Bridge — environments around `hathor-core`

A virtual environment is the container every other Track C tool fills. Forward-pointers:

Bridge — virtual environments in the project and the stack:

- **It manipulates `sys.path`.** The environment's whole job is to put its private `site-packages` on the import search path of Chapter 11 (§11.5) — that is the mechanical link between these two chapters — **Chapter 11**.
- **Poetry manages the environment.** `hathor-core` does not ask you to create or activate a `.venv` by hand; **Poetry** creates one and installs the locked dependency set into it. The environment is the destination; Poetry is the manager — **Chapter 13**, next.
- **The lock file makes it reproducible.** The exact versions filling the environment are pinned in `poetry.lock`, so the same environment can be rebuilt anywhere — **Chapter 13**.
- **Docker is an environment taken to the extreme.** A container isolates not just Python packages but the entire operating environment (system libraries like RocksDB's, the interpreter, the app) — the same isolation instinct, one level down — **Chapter 15**.
- **Why this isolation matters for a node.** Running mainnet infrastructure demands that the software run identically on the developer's laptop and the production server; environments (then Poetry, then Docker) are the layered answer to that demand — recurring through **Chapters 13 & 15**.

Recap

Concept	What it is	Why it matters
Dependency hell	projects fighting over one global package pile	the problem environments solve
site-packages	the directory installed packages live in	each environment has its own
<code>.venv</code>	a directory: own <code>python</code> + own <code>site-packages</code>	the isolation, made concrete
the env's <code>python</code>	a symlink to a real interpreter, own search path	isolation lives here, not in activation
activation	prepends <code>.venv/bin</code> to <code>PATH</code>	convenience; optional
<code>pip</code> / <code>PyPI</code>	installer / package repository	fills an environment
reproducibility	rebuild an identical environment elsewhere	needs a lock file → Poetry (Ch 13)

A virtual environment is not magic — it is a directory holding a symlinked Python and a private `site-packages`, whose entire effect is to swap which packages the import system of Chapter 11 can find. It cures dependency hell by giving each project its own isolated pile, keeps the system Python untouched, and lays the groundwork for reproducibility. Activation is a shell convenience you can skip; the real isolation is in which interpreter you run. What a bare environment lacks is a record of exactly which versions belong in it — and that gap is precisely what the next chapter fills: **Poetry**, the tool that declares, locks, and installs `hathor-core`'s dependencies into an environment it manages for you.

-
1. A virtual environment is an isolated, self-contained Python setup for one project — a directory with its own interpreter link and its own package directory — so each project's dependencies stay separate from other projects' and from the system Python. ↵
 2. `site-packages` is the directory where third-party (pip-installed) packages are placed. There is a global one for the system interpreter and a private one inside each virtual environment; which is used depends on which `python` runs. ↵
 3. Dependency hell is the situation where multiple projects (or libraries) require conflicting versions of the same package, making it impossible to satisfy them all from a single shared installation. ↵
 4. A symbolic link (symlink) is a filesystem entry that points to another file or directory — like a shortcut. A virtual environment's `python` is typically a symlink to a real interpreter, so the environment needn't copy all of Python. ↵
 5. `PATH` is an environment variable holding an ordered list of directories the shell searches to resolve a typed command name. Activating a virtual environment prepends its `bin/` so the environment's `python` / `pip` are found first. ↵
 6. `pip` is Python's standard package installer. It downloads packages (by default from PyPI) and installs them into the active environment's `site-packages`. Poetry uses the same underlying mechanisms but adds dependency resolution and locking. ↵
 7. PyPI (the Python Package Index, at pypi.org) is the central public repository of third-party Python packages that `pip` downloads from by default. ↵